

C S C 2 4 1

Python Programming Language I

Study Manual

Victor Ayodeji

```
>>> total = 0
>>> for n in scores:
    total += n
>>> print(total / len(scores))

72.5 ■
```

How to use this manual

Read this page once. It is short, and it decides whether the rest of the manual works for you.

TEACH

THE BARGAIN

This manual assumes you have never written a line of code, and it carries you from there to a full mark in the examination. It is also meant to be **enough on its own**. You should never need a lecturer, a website or the official documentation to follow it. Anything it uses, it teaches first.

In exchange it asks one thing: **type the code**. Reading code creates a feeling of understanding that disappears in the examination hall. Typing it, getting it wrong, and fixing it is the thing that actually teaches. Every code block here was run on a real interpreter before it was printed, so if yours does something different, the difference is information.

The boxes

Every page is built from a small set of boxes. Each has one job, and the colour tells you which before you read a word.

TEACH

New ideas, explained from nothing. Read these properly.

● MUST MEMORISE

Python yellow means **learn this by heart**. It is the only thing this colour is ever used for in this manual. Nothing decorative is yellow. If a page has yellow on it, there is something on that page you are expected to know cold.

WORKED

A problem solved in numbered steps, then a **now you try** at the end. Do the "now you try" before moving on.

DRY RUN

A program traced by hand, one line at a time, with a table of what every variable holds. This examination asks you to state what code displays, and this is how you work it out without a computer. **Dry run** is the term used in this course.

⚠ TRAP

A mistake that is easy to make and costly. Most of these are taken from the errors the examination itself plants in its code.

RECALL

Questions with the answers printed underneath, below a dashed line. **Cover the answers**. Say your answer out loud, or write it, and only then look. Checking whether you knew something is not the same as knowing it, and the difference is the whole point of this box.

Reading the code

● MUST MEMORISE

THREE KINDS OF BLOCK

Code appears in three kinds of block, and confusing them is the most common early mistake. Each has a different coloured left edge.

1. The interactive prompt. Lines starting `>>>` are what you type. Lines without it are what Python printed back. You never type the `>>>` itself.

```
>>> 2 + 2
4
```

2. A script. The contents of a file, to be saved and run as a whole. No `>>>` anywhere.

```
print("saved in a file")
```

3. Output. What appeared on screen. You never type this.

```
saved in a file
```

Where a keystroke is meant rather than something to type, it is shown as a key: press `Enter`.

TEACH

ONE PROMISE ABOUT OUTPUT

Every output shown in this manual was captured from a real Python 3.13 interpreter at the moment this page was built. None was typed by hand from memory. This matters more than it sounds: a printed output that was never actually run is just a confident guess, and where Python's real answer is surprising, a guess is exactly where it will be wrong. You will meet one such surprise in Module Two, Unit 2, and it is the reason for this promise.

MODULE ONE

Introduction to Python Programming

What a program actually is, what the machine does with it, and how to get Python running on your own computer so that every page after this one is something you can try rather than something you read.

```
>>> print( "hello" )
```

```
hello
```

Overview of Python

Start here even if you have never written a line of code. This unit assumes nothing.

By the end of this unit you can: say what a program is, explain what an interpreter does and how it differs from a compiler, state why Python is used, and name the places Python is used in practice.

What a program is

TEACH

FROM ZERO

A computer does nothing on its own. It waits to be told, in exact terms, what to do. A **program** is that telling: an ordered list of instructions, written down, that the computer carries out one after another.

Think of a recipe. A recipe is not the cooking; it is the written instructions that produce the cooking when somebody follows them. A program is not the work; it is the written instructions that produce the work when the computer follows them.

Two words that follow from this, and that you will use constantly:

- ▶ **Code** is the text you write. The instructions themselves.
- ▶ **Running** (or **executing**) a program means telling the computer to carry out those instructions now, from the top, in order.

Why we need a language

TEACH

The computer's own vocabulary is numbers. Nothing else. Instructions inside the machine are patterns of the digits 0 and 1, called **machine code**. Writing anything useful directly in 0s and 1s is possible and unbearable.

So we write in a **programming language** instead: a fixed, small, precisely defined set of words and symbols that a human can read and that can be turned into machine code automatically. Python is one such language.

A programming language is deliberately far stricter than English. In English, "print hello" and "hello, print it" and "could you print hello" all land. A programming language accepts exactly one form and rejects the rest. That strictness is not the language being difficult. It is the reason the translation to machine code can be done without a human guessing at your intention.

Interpreter and compiler

Something must sit between your text and the machine's numbers. There are two ways to build that something, and the difference is examinable.

TEACH

EXAMINABLE

A **compiler** reads your *whole* program, translates all of it into machine code, and writes that machine code out as a new file. Nothing runs during translation. You run the produced file afterwards, and it runs on its own.

An **interpreter** reads your program *one instruction at a time*, and carries out each instruction as soon as it has read it. Nothing is written out. There is no separate file to run afterwards. Translation and running are the same activity.

Python is interpreted. The program that reads your Python and does what it says is called **the Python interpreter**, and installing Python (Unit 2) means installing it.

● MUST MEMORISE

COMPILER VS INTERPRETER

	COMPILER	INTERPRETER
How much it reads before acting	The whole program	One instruction
Produces a machine code file?	Yes	No
When errors surface	All at translation, before any run	When that line is reached
Speed of the finished program	Faster	Slower
Example language	C, C++	Python

Marks are lost here by naming only one difference. The question asks you to differentiate, so give the pair each time: what it reads, and what it produces.

⚠ TRAP

"Interpreted means Python has no errors until it runs" is **half true and the half that is false costs marks**. Python still checks that each line is *grammatically* possible before running anything in that file. A missing colon is caught immediately, before the first line executes. What waits until the line is reached is a *meaning* error, such as dividing by zero.

You will meet this split again as `SyntaxError` versus runtime error in Module Two, and it is worth marks on its own.

Why Python

TEACH

- ▶ **It reads close to English.** Fewer symbols stand between your intention and the page, so there is less to get wrong.
- ▶ **It is short.** Work that takes many lines elsewhere often takes one here.
- ▶ **It comes with a large standard library:** a big collection of ready made code that ships with Python, so common jobs are already solved. You will use one of these, `sqlite3`, in Module Five.
- ▶ **It is portable.** The same Python file runs on Windows, macOS and Linux.
- ▶ **It is free** and its interpreter is open source.

TEACH

WHERE PYTHON IS USED

FIELD	WHAT PYTHON DOES THERE
Web development	Runs the server behind a website
Data analysis	Reads large data files, summarises and charts them
Artificial intelligence	Builds and trains models
Automation and scripting	Repetitive machine work, such as renaming thousands of files
Scientific computing	Simulation and numerical work
Software testing	Automatically exercising other programs

1. What is a program?
2. Give the two differences between a compiler and an interpreter that earn the marks.
3. Is Python compiled or interpreted?
4. Name three fields where Python is used.

ANSWERS

1. An ordered list of instructions that a computer carries out, written down in a language it can be made to understand.
2. A compiler reads the whole program and produces a machine code file. An interpreter reads one instruction at a time and carries it out at once, producing no file.
3. Interpreted.
4. Any three of: web development, data analysis, artificial intelligence, automation, scientific computing, software testing.

MODULE ONE · UNIT 2

Setting Up Python

Do this once, on your own machine, before you read Module Two. Everything after here is meant to be typed, not watched.

By the end of this unit you can: install Python, confirm it installed, use the interactive prompt, write and run a script file, and say which of the two you are looking at.

Installing

TEACH

WINDOWS

1. Open `python.org` and choose Downloads. The site offers you the installer for the system you are on.
2. Run the installer.
3. **Tick "Add Python to PATH" on the first screen.** This is the one step that people skip and then cannot proceed. It is explained directly below.
4. Choose "Install Now" and wait for it to finish.

● MUST MEMORISE

PATH

PATH is a list your operating system keeps of the folders it will search when you type a command. If Python's folder is not on that list, then typing `python` gets you "command not found", even though Python is installed perfectly well. The program is there; the system just does not know where to look.

If you have already installed without ticking it, run the installer again, choose Modify, and add it. You do not need to uninstall anything.

Both usually ship with a Python already, but often an old one. Install the current version from `python.org` in the same way. On Linux you may instead use your system's package manager, for example `sudo apt install python3` on Ubuntu.

On macOS and Linux the command is often `python3` rather than `python`. If `python` gives you an error or a version starting with 2, use `python3` everywhere in this manual.

Confirming it worked

Open a **terminal**, the window where you type commands to your operating system rather than clicking. On Windows it is called Command Prompt or PowerShell; on macOS and Linux it is called Terminal. Type this and press **Enter**:

IN THE TERMINAL, NOT IN PYTHON

```
python --version
```

It should answer with a version number:

```
Python 3.13.5
```

Your exact numbers will differ, and that is fine. Any version starting with **3** is fine. If it starts with 2, or you get "command not found", see the PATH box above, and try `python3 --version`.

If your version starts with **2**, stop and fix it. Python 2 is a different language in small, silent ways. In Python 2, `5 / 2` is `2`. In Python 3 it is `2.5`. Nothing warns you; you simply get wrong answers. Every page of this manual is Python 3.

The two ways to run Python

This is the distinction that confuses beginners more than any other, so it gets its own box. From here on, every code block in this manual is labelled as one or the other.

1. The interactive prompt (also called the REPL, or interactive mode). You type one line, press **Enter**, and Python answers immediately. You recognise it by the three symbols `>>>` at the left, which Python prints to mean "your turn". It forgets everything when you close it.

2. A script. You type many lines into a file, save it with a name ending in `.py`, and run the whole file at once. It keeps everything, because it is a file.

The prompt is for asking. A script is for building. Your exam answers are scripts.

Type `python` on its own in the terminal and press **Enter**. The prompt appears:

```
>>>
```

Now type an instruction and press **Enter**. Python answers on the next line, then offers you the prompt again:

```
>>> print("Hello, World!")
Hello, World!
>>>
```

To leave the prompt and return to the terminal, type `exit()` and press **Enter**.

● MUST MEMORISE

READING TRANSCRIPTS

In every prompt transcript in this manual:

- ▶ Lines beginning `>>>` are what **you type**. Do not type the `>>>` itself; Python prints that.
- ▶ Lines with no `>>>` are what **Python printed back**.

So in the transcript above, you typed 21 characters and Python printed `Hello, World!`. Getting this backwards is the single most common early confusion, which is why the two are given different coloured edges throughout.

Open any plain text editor. Notepad works. So does IDLE, which was installed alongside Python: find it in your programs, then choose File, then New File.

Type these two lines:

HELLO.PY

```
print("Hello, World!")
print("I am running from a file.")
```

Save it as `hello.py`. The `.py` ending is what marks a file as Python. Then, in the terminal, move to the folder you saved it in and type:

IN THE TERMINAL

```
python hello.py
```

Python reads the file from the top, carries out each line in order, prints:

```
Hello, World!
I am running from a file.
```

and then exits. Notice there is no `>>>` anywhere: you are not having a conversation, you are handing over a finished list of instructions.

Typing `python hello.py` while already at the `>>>` prompt gives you a `SyntaxError`. Those are terminal commands, not Python instructions. They live in different places:

```
>>> python hello.py
File "<stdin>", line 1
  python hello.py
    ^^^^^
SyntaxError: invalid syntax
```

If you see this, you are at the Python prompt when you meant to be at the terminal. Type `exit()` first.

1. What does `PATH` do, and what goes wrong if Python is not on it?
2. What do the three symbols `>>>` tell you?
3. Which ending must a Python file's name have?
4. You want to keep your work and hand it in. Prompt or script?

ANSWERS

1. `PATH` lists the folders the operating system searches when you type a command. If Python's folder is missing from it, typing `python` reports "command not found" even though Python is installed.
2. That you are at the interactive prompt, and that Python is waiting for you to type one instruction. You never type the symbols yourself.
3. `.py`
4. A script. The prompt forgets everything when it closes.

MODULE TWO

Python Basics and Syntax

The grammar of the language, the boxes you keep values in, and the symbols that combine them. Everything in the rest of the manual is built out of what is in these three units.

```
age = 25
```

```
name = "Ada" ■
```

Python Language Syntax

Syntax means the grammar: the rules that decide whether a line is Python at all, before anyone asks whether it is correct.

By the end of this unit you can: write and run statements, use comments, explain why indentation is not decoration in Python, print with `sep` and `end`, use escape characters, format with f-strings, and read a traceback.

Statements

TEACH

A **statement** is one complete instruction. In Python, one statement occupies one line, and the end of the line ends the statement. There is no symbol to mark the end.

```
print("first")
print("second")
```

Two lines, two statements, carried out top to bottom. If you know a language that ends statements with a semicolon, note that Python does not want one. It is not an error, but it marks your code as written by somebody thinking in another language, and the marks in this course go to Python style.

Comments

TEACH

A **comment** is a note to a human. Python ignores it completely. Everything from a `#` to the end of that line is a comment.

```
# Work out the area of the room.
area = length * width      # in square metres
```

Comments do not run, do not print, and do not affect anything. They exist so that you, in three weeks, can tell what you meant.

● MUST MEMORISE

READ THIS ONCE, CAREFULLY

A comment is **never** output. If you see

```
>>> 10 % 2
0      # Even
```

then the output is `0`. The words `# Even` are a human annotation sitting beside it, not something Python said. Python never answers you in comments.

This matters because printed material sometimes puts the annotation where the output should be, and a reader new to the language cannot tell which is which. In this manual the rule is absolute: **output is shown on its own line, in an output block, exactly as a real interpreter printed it.** Every output in this manual was captured from a run, not typed by hand.

Indentation

TEACH

PYTHON'S ONE BIG IDEA

In most languages, spaces at the start of a line are decoration, and blocks of code are marked with braces. **In Python the spaces are the structure.** There are no braces. Indentation is how Python knows which lines belong to what.

You have not met `if` yet, and it arrives properly in Module Three. Read this only for the shape:

```
if raining:
    print("take a coat")      # indented: belongs to the if
print("leave the house")    # not indented: always runs
```

The indented line runs only when it is raining. The unindented line always runs. Move that one line four spaces to the left or right and you have changed what the program does. Nothing else changed.

The rule: a colon at the end of a line means "a block follows", and that block is the lines indented beneath it. Use **four spaces** per level. Be consistent: Python accepts any consistent width, and rejects a mixture.

⚠ TRAP

EXAM FAVOURITE

Indentation errors are a standing question in this course. Two distinct failures, and the wording of the error tells you which:

```
IndentationError: expected an indented block after 'if' statement on line 1
```

means you wrote the colon and then did not indent anything beneath it.

```
IndentationError: unexpected indent
```

means you indented a line that had no colon above it to belong to.

Printing

TEACH

`print()` displays things. Give it several items separated by commas and it prints them all, putting **one space** between each:

```
>>> print("Ada", 25, "Lagos")
Ada 25 Lagos
```

It then moves to a new line. Both of those behaviours can be changed, and each has a name you must know.

`sep` sets what goes **between** the items. Default: one space.

`end` sets what goes **after** the last item. Default: a newline, written `\n`.

```
>>> print("2026", "07", "17", sep="-")
2026-07-17
>>> print("Ada", "Bob", sep="")
AdaBob
```

Setting `end` to the empty string stops the newline, so the next `print` continues on the same line:

```
>>> print("Hello", end="")
>>> print("World")
HelloWorld
```

Read that transcript carefully. The first `print` displayed `Hello` and stayed put; the second added `World` and ended the line. The output `HelloWorld` is the work of both statements together.

Quotes and escape characters

TEACH

Text in Python is written inside quotes, and is called a **string**. Single and double quotes are equally valid and mean exactly the same thing:

```
'Ada'
"Ada"
```

Having both is useful, because a string in double quotes may freely contain a single quote:

```
>>> print("It's fine")
It's fine
```

The problem arrives when the string must contain the same quote that ends it. Python reads the second quote as the end of the string and the rest becomes nonsense. The fix is an **escape character**: a backslash `\` before the quote, which tells Python "this next character is data, not punctuation".

```
>>> print("She said, \"Python is cool!\"")
She said, "Python is cool!"
```

● MUST MEMORISE

ESCAPE CHARACTERS

ESCAPE	MEANS
<code>\n</code>	Newline: move to the next line
<code>\t</code>	Tab: jump to the next tab stop
<code>\"</code>	A double quote, as data
<code>\'</code>	A single quote, as data
<code>\\</code>	One backslash, as data

A question asking you to "list three escape characters and their effects" has appeared in this course. The three safest to give are `\n`, `\t` and `\\`.

TEACH

ONE STRING, TWO LINES

`\n` inside a string breaks it across lines when printed. It is one string, not two:

```
>>> print("Name: Ada\nCity: Lagos")
Name: Ada
City: Lagos
```

f-strings

TEACH

You will constantly want to put a value inside a sentence. The direct way is an **f-string**: put the letter `f` immediately before the opening quote, then write `{ }` wherever a value belongs.

```
>>> name = "Ada"
>>> age = 25
>>> print(f"{name} is {age}")
Ada is 25
```

The `f` is not optional and not decoration. Without it you get the braces themselves:

```
>>> print("{name} is {age}")
{name} is {age}
```

After a colon inside the braces you may say *how* to display the value. These are the ones this course uses:

WRITTEN	MEANS	EXAMPLE IN	EXAMPLE OUT
:.2f	Fixed to 2 decimal places	123.456	123.46
:,	Comma between thousands	1250000	1,250,000
:,.2f	Both together	9876543.21	9,876,543.21
:.2%	As a percentage, 2 decimals	0.875	87.50%
:<10	Left aligned, width 10	'Item'	Item.....
:>10	Right aligned, width 10	'Price'	Price

In the last two rows each dot stands for one space, so that you can count them. Python prints spaces, not dots.

WORKED

ALIGNMENT YOU CAN ACTUALLY SEE

Field widths exist to build columns, and columns only work in a font where every character takes the same width. Every code block in this manual is set in such a font. Give three rows **the same format string** and they line up:

```
>>> print(f"{'Item':<10}{'Price':>10}")
Item          Price
>>> print(f"{'Apple':<10}{1.5:>10.2f}")
Apple         1.50
>>> print(f"{'Banana':<10}{12.75:>10.2f}")
Banana        12.75
```

The words start in the same column because `:<10` pins them to the left. The prices **end** in the same column because `:>10` pins them to the right, and that is true even though 1.50 and 12.75 have different lengths. That is what a field width buys you.

Spaces cannot be counted by eye, so here are those same three lines with every space drawn as a dot. Each dot is one space, and nothing else has changed:

```
Item.....Price
Apple.....1.50
Banana.....12.75
```

Count the first line. Item takes 4 of its 10 columns, so **6** spaces pad it on the right. Price is 5 characters pushed to the right hand edge of its own 10 columns, so **5** spaces come first. That is $6 + 5 = 11$ dots, which is what you counted. Every line is 20 characters wide.

Now you try: work out the dots for a row holding 'Fig' and 3.5 in the same format, before you run it. (3 + 7 pad, then 6 + '3.50': Fig then 13 dots then 3.50.)

Reading a traceback

TEACH

WORTH MANY MARKS

When Python cannot do what you asked, it prints a **traceback**. Beginners see a wall of red and look away. It is the most useful text on the screen, and half of this course's code marks come from finding errors, so learn to read it.

```
Traceback (most recent call last):
  File "C:\Users\you\hello.py", line 3, in <module>
    print(10 / 0)
    ~~~^~~
ZeroDivisionError: division by zero
```

Read it **from the bottom up**:

- ▶ **Last line** is what went wrong: `ZeroDivisionError`, and in plain words, division by zero. This is the line that tells you the problem.
- ▶ **Above it** is the offending code, with the marks `~~~^~~` pointing at the exact part that failed. The `^` sits under `/`, the operation that could not be done.
- ▶ **Above that** is where: the full path of the file, and line 3.

Error type, then message, then place. Three facts, always in that order.

Python prints the file's whole location, so your first line will be longer than the one above and will show wherever you saved it. Only the file name and the line number matter to you.

● MUST MEMORISE

SYNTAX BEFORE RUNTIME

Errors come in two kinds and they happen at **different times**:

A `SyntaxError` is a grammar failure. Python finds it while reading the file, *before running a single line*. Nothing in the file executes. There is no traceback of the kind above, because nothing ran.

A runtime error is a meaning failure in a line that is grammatically fine. Python only finds it when it reaches that line, so everything before it has already run and printed.

The consequence, which is examinable: a `SyntaxError` anywhere in a file hides every other error in it. If a file has a missing bracket on line 2 and a misspelled name on line 9, you are told about line 2 only. Fix it, run again, and only then hear about line 9.

RECALL

COVER THE ANSWERS

1. What does a colon at the end of a line announce?
2. What are `sep` and `end`?
3. Which end of a traceback do you read first?
4. A file has a `SyntaxError` on line 2 and a `NameError` on line 9. What does Python report?

ANSWERS

1. That a block follows, made of the lines indented beneath it.
2. `sep` is what `print` puts between items (default one space); `end` is what it puts after the last one (default a newline).
3. The bottom. The last line names the error and states it in words.
4. The `SyntaxError` on line 2, and nothing else. The file never runs, so line 9 is never reached.

Variables and Data Types

Where values are kept, what kinds of value exist, and the one thing about numbers that surprises everybody exactly once.

By the end of this unit you can: create and reassign variables, name them legally, list the four basic types, check a type, read input, convert between types, and explain why `0.1 + 0.2` is not `0.3`.

Variables

TEACH

A **variable** is a name that refers to a value. You make one by writing the name, then `=`, then the value:

```
>>> age = 25
>>> age
25
```

The `=` here does **not** mean equals. It means *assign*: work out the value on the right, then make the name on the left refer to it. It is an instruction, not a statement of fact. This is why

```
count = count + 1
```

is perfectly sensible, though as mathematics it would be nonsense. It reads: take what `count` is now, add 1, and make `count` refer to that.

TEACH

REASSIGNMENT

A variable can be given a new value at any time, including a value of a different kind. The old value is simply forgotten:

```
>>> x = 99
>>> print(x)
99
>>> x = "Take me to your leader."
>>> print(x)
Take me to your leader.
```

Python does not object. The name is a label, not a box of a fixed size.

● MUST MEMORISE

NAMING RULES

Must:

- ▶ Start with a letter or an underscore. **Never** a digit.
- ▶ Contain only letters, digits and underscores. No spaces, no hyphens.
- ▶ Not be a Python keyword (`if`, `for`, `class`, and the rest).

Should: be lowercase with underscores between words (`total_score`), and say what it holds. A name like `x` tells the reader nothing.

Case matters. `age`, `Age` and `AGE` are three different variables. This is a standing source of exam errors: a program that defines `square` and calls `Square` is broken.

The four basic types

● MUST MEMORISE

TYPES

TYPE	NAME IN PYTHON	WHAT IT HOLDS	EXAMPLES
Integer	int	Whole numbers, no decimal point	25, 0, -7
Float	float	Numbers with a decimal point	3.14, -0.5, 2.0
String	str	Text, in quotes	"Ada", 'x', ""
Boolean	bool	Truth, one of exactly two values	True, False

True and False are capitalised and are not strings. "True" in quotes is a string.

⚠ TRAP

2.0 IS NOT AN INT

2 and 2.0 are equal in value but are different types. The decimal point makes it a float, even when there is nothing after it. This decides what division gives you back, as Unit 3 shows.

TEACH

TYPE()

type() tells you what something is. When in doubt, ask:

```
>>> type(25)
<class 'int'>
>>> type(3.14)
<class 'float'>
>>> type("Ada")
<class 'str'>
>>> type(True)
<class 'bool'>
```

Read <class 'int'> as "this is an int". The word class is Python's internal vocabulary for a type.

Input

● MUST MEMORISE

INPUT ALWAYS GIVES A STRING

input() stops the program, waits for the user to type something and press **Enter**, and hands back what they typed. The text you pass to it is shown as the prompt.

What it hands back is always a string. Always. Even when the user types digits.

```
age = input("How old are you? ")
print(type(age))
```

```
How old are you? 25
<class 'str'>
```

The user typed 25 and Python is holding the two characters "25", not the number. This single fact causes more broken exam answers than anything else in the course.

Forget to convert, and arithmetic does something silently absurd rather than failing:

```
a = input("First number: ") # user types 5
b = input("Second number: ") # user types 3
print(a + b)
```

```
First number: 5
Second number: 3
53
```

53, not 8. Because `+` on two strings joins them end to end. The program did exactly what you asked and nothing reported an error. The fix is to convert.

Converting between types

● MUST MEMORISE

CASTING

FUNCTION	TURNS A VALUE INTO	EXAMPLE	RESULT
<code>int()</code>	an integer	<code>int("25")</code>	25
<code>float()</code>	a float	<code>float("3.5")</code>	3.5
<code>str()</code>	a string	<code>str(100)</code>	'100'

So the fix for the trap above, and the pattern you will write hundreds of times:

```
age = int(input("How old are you? "))
```

Read it from the inside out: `input` collects text, `int` turns that text into a number, `=` stores the number.

TEACH

INT() TRUNCATES

`int()` on a float throws the decimal part away. It does not round:

```
>>> int(3.9)
3
>>> int(-3.9)
-3
```

It cuts towards zero. Use `round()` if you want rounding.

⚠️ TRAP

INT() IS FUSSY ABOUT STRINGS

`int()` can only convert a string that is entirely a whole number. Give it anything else and you get a runtime error, and the program stops:

```
>>> int("xyz")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'xyz'
```

Note that `int("3.5")` fails too, for the same reason: `3.5` is not a whole number written out. Use `float("3.5")`, or `int(float("3.5"))` if you truly want `3`.

Floats are not exact

● MUST MEMORISE

EVERYBODY MEETS THIS ONCE

Type this at the prompt. It is not a trick:

```
>>> 0.1 + 0.2
0.30000000000000004
```

Python is not broken and you did not make a mistake. A computer stores numbers in binary, in a fixed amount of space. Some numbers that are short and exact in decimal, such as 0.1, cannot be written exactly in binary at all, in the same way that 1/3 cannot be written exactly in decimal: 0.333... never finishes. Python stores the closest value it can fit, and the tiny difference shows up when you add.

This is not a Python quirk. It is how nearly every language on nearly every machine stores decimals.

WORKED

WHERE IT WILL BITE YOU

It hides until it does not. Watch an ordinary looking interest calculation:

```
>>> INTEREST_RATE = 0.069
>>> balance = 10000
>>> amount = balance * INTEREST_RATE
>>> amount
690.00000000000001
```

Not 690.0. And yet change the rate to 0.072 and it comes out perfectly clean:

```
>>> INTEREST_RATE = 0.072
>>> amount = balance * INTEREST_RATE
>>> amount
720.0
```

Same code, same shape, one is ugly and one is clean. That is exactly what makes this dangerous: you cannot predict which by looking, so you must handle it always rather than when it looks likely.

The rule: never show a raw float to a user, and never compare two floats with `==`. Format it on the way out.

● MUST MEMORISE

THE FIX

Format the float when you display it, and the problem disappears at the only point where it mattered:

```
>>> amount = 10000 * 0.069
>>> print(f"Interest: {amount:.2f}")
Interest: 690.00
```

Any exam program that reports an average, a percentage or an amount of money should print it through `:.2f`. It is one keystroke's worth of effort and it is the difference between a professional answer and a puzzling one.

1. What type does `input()` return?
2. The user types 5 and 3 into two `input()` calls and you print `a + b`. What appears, and why?
3. What is `int(3.9)`?
4. Why is `0.1 + 0.2` not `0.3`?

ANSWERS

1. A string, always, even if the user typed digits.
2. 53. Both are strings, and `+` on strings joins them. No error is reported.
3. 3. `int()` truncates towards zero, it does not round.
4. Floats are stored in binary in fixed space. 0.1 has no exact binary form, so Python stores the nearest value it can, and the small error becomes visible when adding. Format with `:.2f` on output.

MODULE TWO · UNIT 3

Python Operators

The symbols that combine values. Two of them, `//` and `%`, carry more exam marks than the rest put together.

By the end of this unit you can: use every arithmetic operator, predict what division returns, use `%` to test divisibility, apply precedence correctly, and use comparison and logical operators.

Arithmetic

● MUST MEMORISE

ALL SEVEN

OPERATOR	NAME	EXAMPLE	RESULT
+	Add	<code>7 + 5</code>	12
-	Subtract	<code>7 - 5</code>	2
*	Multiply	<code>7 * 5</code>	35
/	Divide	<code>7 / 5</code>	1.4
//	Floor divide	<code>7 // 5</code>	1
%	Remainder (modulus)	<code>7 % 5</code>	2
**	Exponent (power)	<code>7 ** 2</code>	49

● MUST MEMORISE

/ ALWAYS GIVES A FLOAT

/ gives a **float every time**, even when the division is exact:

```
>>> 10 / 2
5.0
>>> 5 / 2
2.5
```

$10 / 2$ is 5.0 , not 5 . If you need a whole number, use `//` or wrap it in `int()`.

`//` divides and throws away the fractional part, giving the whole number of times the divisor fits:

```
>>> 7 // 5
1
>>> 17 // 3
5
```

● MUST MEMORISE

% IS THE WORKHORSE

`%` gives the **remainder** after division. $17 \% 3$ is 2 , because 3 fits into 17 five times with 2 left over.

`//` and `%` are the two halves of one division:

```
>>> 17 // 3
5
>>> 17 % 3
2
```

17 divided by 3 is **5 remainder 2**. The first gives the 5, the second the 2.

The use that appears in this exam every year: a number is divisible by `n` exactly when `number % n == 0`.

```
>>> 10 % 2
0
>>> 7 % 2
1
```

A remainder of 0 on division by 2 means even. A remainder of 1 means odd. Read those two transcripts again: the outputs are `0` and `1`. That is what Python printed.

This is `//` and `%` working together, and it is the standard demonstration of both. Turn 11730 seconds into hours, minutes and seconds.

- 1 An hour is 3600 seconds. **How many whole hours fit?** That is floor division.

11730 // 3600 gives 3

- 2 **What is left over after those hours?** Take the remainder on 3600, then ask how many whole minutes fit in it. Both steps at once: take the remainder on 60 of the total whole minutes.

(11730 // 60) % 60 gives 15

- 3 **Seconds left over** is the remainder on 60.

11730 % 60 gives 30

TIME_CONVERTER.PY

```
total_seconds = 11730
hours = total_seconds // 3600
minutes = (total_seconds // 60) % 60
seconds = total_seconds % 60
print("Hours:", hours)
print("Minutes:", minutes)
print("Seconds:", seconds)
```

```
Hours: 3
Minutes: 15
Seconds: 30
```

Now you try: change 11730 to 7325 and predict the three numbers before running it. (2 hours, 2 minutes, 5 seconds.)

● MUST MEMORISE

**** IS RIGHT ASSOCIATIVE**

Nearly every operator groups from the left. `**` groups from the **right**:

```
>>> 2 ** 3 ** 2
512
```

This is `2 ** (3 ** 2)`, which is `2 ** 9`, which is 512. It is **not** `(2 ** 3) ** 2`, which would be 64.

`**` also does roots, because a square root is the power one half:

```
>>> 9 ** 0.5
3.0
```

Precedence

● MUST MEMORISE

ORDER OF OPERATIONS

When several operators appear in one expression, Python applies them in this order, highest first:

ORDER	OPERATORS
1	() brackets
2	**
3	* / // %
4	+ -

Within one row, work left to right (except **, which goes right to left).

```
>>> 2 + 3 * 4
14
>>> (2 + 3) * 4
20
```

When in doubt, add brackets. They cost nothing and they make your intention unmistakable to a marker.

Comparison operators

● MUST MEMORISE

THESE GIVE TRUE OR FALSE

OPERATOR	MEANS	EXAMPLE	RESULT
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	7 > 5	True
<	Less than	4 < 10	True
>=	Greater than or equal to	7 >= 7	True
<=	Less than or equal to	2 <= 4	True

⚠ TRAP

= AGAINST ==

= assigns. == asks. They are unrelated, and confusing them is the most common single error in this course's error-spotting questions.

```
age = 18      # assigns 18 to age
age == 18     # asks: is age 18? Gives True.
```

Write `if age = 18:` and Python stops you with a `SyntaxError`, because a question was expected and you gave it an instruction.

Python lets you write a range check the way mathematics does, and it means what it looks like:

```
>>> bmi = 22.4
>>> 18.5 <= bmi <= 24.9
True
```

This is worth remembering: a question in this course asks for exactly this test on a BMI value, and the chained form is shorter and clearer than joining two comparisons with `and`.

Logical operators

● MUST MEMORISE

AND, OR, NOT

OPERATOR	TRUE WHEN	EXAMPLE	RESULT
<code>and</code>	Both sides are true	True and False	False
<code>or</code>	Either side is true	True or False	True
<code>not</code>	The other way round	not True	False

These are English words in Python, not symbols. If you have seen `&&` and `||` elsewhere, they do not exist here.

```
>>> n = 15
>>> n % 3 == 0 and n % 5 == 0
True
```

Fix that line in your memory. It asks "is n divisible by both 3 and 5", and it is the first test in the FizzBuzz style question this course sets.

Assignment operators

● MUST MEMORISE

SHORTHAND

Changing a variable using its own current value is so common it has shorthand:

SHORTHAND	MEANS
<code>x += 1</code>	<code>x = x + 1</code>
<code>x -= 1</code>	<code>x = x - 1</code>
<code>x *= 2</code>	<code>x = x * 2</code>
<code>x /= 2</code>	<code>x = x / 2</code>

`x += 1` is read "increment x". You will write it in every counting loop in Module Three, and forgetting it is what makes a loop run forever.

DRY RUN

TRACE IT BY HAND

Tracing a program by hand, one line at a time, writing down what each variable holds, is the skill this course tests as "what will the following code display". Practise it now on something small, so that it is automatic later.

```
a = 7
b = 2
a += b
c = a // b
d = a % b
print(a, b, c, d)
```

Take one line at a time and never look ahead:

LINE	A	B	C	D	OUTPUT
a = 7	7	-	-	-	
b = 2	7	2	-	-	
a += b	9	2	-	-	
c = a // b	9	2	4	-	
d = a % b	9	2	4	1	
print(...)	9	2	4	1	9 2 4 1

A dash means the variable does not exist yet. Using one before it exists is a `NameError`.

CONFIRMED AGAINST A REAL INTERPRETER

```
9 2 4 1
```

RECALL

COVER THE ANSWERS

1. What is `10 / 2`, and what type is it?
2. How do you test whether `n` is divisible by 5?
3. What is `2 ** 3 ** 2`?
4. What is the difference between `=` and `==`?
5. What is `17 // 3` and `17 % 3`?

ANSWERS

1. `5.0`, a float. `/` always gives a float.
2. `n % 5 == 0`
3. `512`. `**` groups from the right, so it is `2 ** 9`, not `(2 ** 3) ** 2`.
4. `=` assigns a value to a name. `==` asks whether two values are equal and gives back `True` or `False`.
5. `5` and `2`. Seventeen is three fives remainder two.